

Metaheurísticas

Guía de trabajos prácticos 4

Algoritmos Evolutivos para Optimización de Redes Neuronales

1. Clasificación de vinos con redes neuronales

El dataset *Wine Recognition*, disponible en la biblioteca `scikit-learn` (`sklearn.datasets.load_wine`), contiene mediciones físico-químicas de 178 muestras de vino obtenidas a partir de tres cultivares distintos. El objetivo es predecir el cultivar (clases 0, 1 y 2) a partir de los $N = 13$ atributos numéricos listados en la Tabla 1. No todas las características son igualmente discriminativas, lo que hace de este dataset un caso de estudio adecuado para evaluar estrategias de selección de variables.

Índice	Característica
1	Alcohol
2	Malic acid
3	Ash
4	Alcalinity of ash
5	Magnesium
6	Total phenols
7	Flavanoids
8	Nonflavanoid phenols
9	Proanthocyanins
10	Color intensity
11	Hue
12	OD280/OD315 of diluted wines
13	Proline

Cuadro 1: Atributos del dataset *Wine Recognition* ($N = 13$).

Para clasificar el cultivar se propone emplear un Perceptrón Multicapa (MLP) implementado mediante la clase `MLPClassifier` de `scikit-learn`. El entrenamiento debe realizarse utilizando el optimizador SGD y la función de activación TanH. La implementación del algoritmo genético deberá realizarse en Python, empleando `numpy` para las operaciones numéricas, `pandas` para el registro de resultados y `matplotlib` para la visualización.

Ejercicio 1: Cargue el dataset *Wine Recognition* mediante `sklearn.datasets.load_wine` y estandarice los atributos de entrada utilizando `StandardScaler`. Reserve el 25% de los datos para el conjunto de prueba, fijando una semilla aleatoria para garantizar la reproducibilidad.

Construya un MLP con la configuración indicada en la Tabla 2 y entrénelo sobre el conjunto de entrenamiento.

Reporte los siguientes resultados:

- La matriz de confusión y el *accuracy* obtenidos sobre el conjunto de prueba.
- La cantidad total de parámetros entrenables del modelo (pesos y sesgos), calculada analíticamente a partir de la arquitectura.

Parámetro	Valor
Arquitectura (capas ocultas)	(16, 8) neuronas
Función de activación	TanH
Optimizador	SGD
Tasa de aprendizaje inicial	0,01
Número máximo de iteraciones	300

Cuadro 2: Configuración del MLP de referencia.

2. Optimización de la arquitectura mediante algoritmos genéticos

El espacio de posibles arquitecturas de un MLP con hasta tres capas ocultas de hasta 128 neuronas cada una es demasiado amplio para ser explorado exhaustivamente. En esta sección se implementa un Algoritmo Genético (AG) para buscar automáticamente una arquitectura que maximice el *accuracy* de validación, utilizando la totalidad de los atributos del dataset como entrada.

Representación del cromosoma

Cada individuo de la población representa una posible arquitectura mediante un cromosoma binario de 23 bits, organizado en tres bloques: un primer bloque de 7 bits correspondiente a la primera capa oculta, y dos bloques de 8 bits para las capas ocultas segunda y tercera.

La primera capa oculta está **siempre activa**: su bit de activación no forma parte del cromosoma y sus 7 bits codifican en binario sin signo el número de neuronas, con la convención $h_1 = \text{bin}(n_1^{(1)} \dots n_1^{(7)}) + 1$, lo que permite un rango de 1 a 128 neuronas. Para las capas 2 y 3, el primer bit (a_l) indica si la capa l está activa ($a_l = 1$) o inactiva ($a_l = 0$), y los siete bits restantes codifican el número de neuronas con la misma convención: $h_l = \text{bin}(n_l^{(1)} \dots n_l^{(7)}) + 1 \in [1, 128]$. Si una capa $l \in \{2, 3\}$ está inactiva, los bits de neuronas de esa capa no se interpretan. La Tabla 3 ilustra la estructura completa del cromosoma.

Capa oculta 1 (bits 1–7)	Capa oculta 2 (bits 8–15)	Capa oculta 3 (bits 16–23)
$n_1^{(1)} \dots n_1^{(7)}$ (activa siempre)	$a_2 \mid n_2^{(1)} \dots n_2^{(7)}$	$a_3 \mid n_3^{(1)} \dots n_3^{(7)}$

Cuadro 3: Estructura del cromosoma binario de 23 bits para la codificación de la arquitectura. La primera capa oculta no tiene bit de activación y está siempre presente.

Ejemplo: el cromosoma $[0000100 \mid 1|0000010 \ 0|0000000]$ representa una red con dos capas ocultas activas: la primera (siempre activa) con $h_1 = 4 + 1 = 5$ neuronas, y la segunda con $h_2 = 2 + 1 = 3$ neuronas. La tercera capa está inactiva ($a_3 = 0$), por lo que sus bits de neuronas se ignoran.

Función de aptitud

La aptitud de cada individuo se mide mediante el *accuracy* promedio evaluado en validación cruzada estratificada de 5 particiones sobre el conjunto de entrenamiento. El uso de validación cruzada, en lugar de evaluar directamente sobre el conjunto de prueba, evita que el proceso de búsqueda sobreajuste a ese conjunto y proporciona una estimación más robusta del rendimiento generalizable de cada arquitectura candidata. La validación estratificada garantiza además que cada partición preserve la distribución de clases del conjunto original, lo que es especialmente relevante cuando las clases no están perfectamente balanceadas:

$$F_1(I) = \text{Accuracy}_{CV}(I)$$

Parámetros del algoritmo genético

El comportamiento de un algoritmo genético depende de manera sensible de su configuración: el tamaño de la población determina el nivel de diversidad inicial, el número de generaciones controla el tiempo de búsqueda, y las probabilidades de cruce y mutación regulan el balance entre explotación y

exploración del espacio de soluciones. Por ello, se espera que los estudiantes exploren distintos valores para cada parámetro y justifiquen las elecciones realizadas en función de los resultados observados. La Tabla 4 propone valores de referencia que constituyen un punto de partida razonable para comenzar la experimentación, pero no deben considerarse fijos ni necesariamente óptimos para este problema.

Parámetro	Valor sugerido
Tamaño de la población	30 individuos
Número de generaciones	100
Probabilidad de cruce	$p_c = 0,8$
Probabilidad de mutación (por bit)	$p_m = 0,01$
Método de selección	Torneo
Tipo de cruce	Punto único

Cuadro 4: Valores sugeridos para los parámetros del algoritmo genético (punto de partida).

Ejercicio 2: Implemente el Algoritmo Genético descrito anteriormente para optimizar la arquitectura del MLP sobre el dataset *Wine Recognition*. Utilice `MLPClassifier` de `scikit-learn` con `solver='sgd'`, `learning_rate_init=0.01`, `activation='tanh'` y `max_iter=300`. Configure los parámetros del AG partiendo de los valores sugeridos en la Tabla 4 y explore variantes que considere relevantes, justificando las decisiones adoptadas.

- Grafique la evolución del mejor *fitness* y del *fitness* promedio de la población a lo largo de las generaciones.
- Identifique el mejor individuo de la última generación, decodifique su cromosoma e informe la arquitectura encontrada.
- Entrene el modelo con esa arquitectura sobre el conjunto de entrenamiento completo y reporte su *accuracy* y la matriz de confusión sobre el conjunto de prueba. Analice la matriz: ¿qué pares de clases concentran la mayor cantidad de errores? ¿El error es simétrico entre clases o hay direcciones de confusión predominantes?
- Calcule la cantidad total de parámetros del mejor modelo y compárela con la del modelo de referencia del Ejercicio 1 de la Sección 1.

3. Optimización multi-objetivo: *accuracy* y complejidad del modelo

Maximizar únicamente el *accuracy* puede conducir a soluciones sobreparametrizadas que usen todas las características disponibles y capas de gran tamaño, sin que ello suponga una mejora significativa en el rendimiento. En esta sección se amplía la representación del cromosoma para incluir la selección de características de entrada, y se introduce un enfoque que permite balancear el rendimiento con la complejidad del modelo.

Representación extendida del cromosoma

El cromosoma se amplía a 36 bits: los primeros 13 bits constituyen una máscara binaria que determina qué atributos del dataset se utilizan como entrada del MLP, y los 23 bits restantes codifican la arquitectura de la red exactamente como se describió en la Sección 2. El bit $m_i = 1$ indica que la característica i está activa y $m_i = 0$ que se excluye. Se define $K = \sum_{i=1}^{13} m_i$ como la cantidad de características activas, y se deberá garantizar $K \geq 1$. La Tabla 5 muestra la estructura del cromosoma extendido.

Bits 1–13	Bits 14–20	Bits 21–28	Bits 29–36
Máscara de características ($m_1 \dots m_{13}$)	Capa oculta 1 ($n_1^{(1)} \dots n_1^{(7)}$, activa siempre)	Capa oculta 2 ($a_2 n_2^{(1)} \dots n_2^{(7)}$)	Capa oculta 3 ($a_3 n_3^{(1)} \dots n_3^{(7)}$)

Cuadro 5: Estructura del cromosoma extendido de 36 bits: 13 bits de máscara de características más 23 bits de arquitectura.

Ejemplo: el cromosoma $[1, 0, 1, 1, 0, 0, 1, 0, 0, 1, 1, 0, 1 \mid 0000100 \mid 1|0000010 \ 0|0000000]$ representa $K = 7$ características activas (índices 1, 3, 4, 7, 10, 11 y 13), una red con la primera capa oculta siempre activa con $h_1 = 5$ neuronas, la segunda capa con $h_2 = 3$ neuronas y la tercera capa inactiva.

Estimación del número de parámetros

Para un individuo I con K características activas y $L \leq 3$ capas ocultas activas con $h_1, h_2, \dots, h_L$ neuronas, el número total de parámetros entrenables del MLP —pesos y sesgos— es:

$$P(I) = (K + 1) h_1 + \sum_{l=2}^L (h_{l-1} + 1) h_l + (h_L + 1) C$$

donde $C = 3$ es el número de clases y el sumando $+1$ en cada factor corresponde a los sesgos. El valor máximo, alcanzado cuando $K = 13$, $L = 3$ y $h_1 = h_2 = h_3 = 128$, es:

$$P_{max} = 14 \cdot 128 + 129 \cdot 128 + 129 \cdot 128 + 129 \cdot 3 = 35\,203$$

Una posible función de aptitud: penalización por complejidad

Abordar simultáneamente el rendimiento y la complejidad del modelo constituye un problema multiobjetivo que admite distintas formulaciones. Una posible función de aptitud que escalariza ambos objetivos en un único criterio es la siguiente función penalizada:

$$F_2(I) = \text{Accuracy}_{CV}(I) - \alpha \cdot \frac{P(I)}{P_{max}}$$

El parámetro $\alpha \geq 0$ regula el balance entre rendimiento y complejidad: valores pequeños de α priorizan el *accuracy*, mientras que valores grandes inducen al AG a preferir modelos más simples con menos características y capas más pequeñas. En el caso límite $\alpha = 0$, la función F_2 coincide con F_1 de la sección anterior. Esta formulación representa una aproximación escalarizada conveniente, pero no es la única forma de plantear el problema. Si lo desea, puede proponer otras formulaciones que capturen el compromiso entre rendimiento y complejidad de manera diferente, justificando la elección y explicando cómo se interpreta el criterio adoptado.

Ejercicio 3: Extienda la implementación del Algoritmo Genético para trabajar con el cromosoma de 36 bits y una función de aptitud que balancee rendimiento y complejidad. Como referencia, puede emplear la función penalizada F_2 descrita anteriormente; si propone una formulación alternativa, justifíquela y explique cómo se interpreta. Explore los valores $\alpha \in \{0,05, 0,1, 0,2, 0,5\}$ (o los parámetros equivalentes de su formulación) y, para cada valor, ejecute el AG y reporte:

- El *accuracy* del mejor individuo sobre el conjunto de prueba.
- La matriz de confusión del mejor individuo sobre el conjunto de prueba.
- El número de características activas K y los índices de las mismas.
- La arquitectura de la red (cantidad de capas activas y neuronas por capa) y el número de parámetros $P(I)$.

Construya una tabla comparativa utilizando **pandas** y analice cómo varía el subconjunto de características seleccionado y la complejidad del modelo a medida que aumenta α . A partir de las matrices de confusión, discuta además si la penalización por complejidad afecta de manera diferente el rendimiento en cada clase, o si los errores se distribuyen de forma similar independientemente de α . Discuta el compromiso entre rendimiento y complejidad, y justifique si el modelo encontrado con algún valor de α resulta preferible al obtenido con F_1 en la Sección 2.

Ejercicio 4: Para estudiar la estabilidad del algoritmo, ejecute 10 veces el AG de la Sección 2 (función F_1) y 10 veces el AG de esta sección con el valor de α que considere más adecuado (función F_2 u

otra formulación equivalente), utilizando distintas semillas aleatorias en cada corrida. Registre los resultados en **DataFrames** de **pandas** e informe:

- Media, desvío estándar, máximo, mínimo y mediana del *accuracy* en el conjunto de prueba para ambas configuraciones (F_1 y F_2).
- Matriz de confusión de la mejor y de la peor solución encontrada en las 10 corridas, tanto para F_1 como para F_2 (es decir, cuatro matrices en total). Analice si las soluciones con mayor y menor *accuracy* difieren también en el tipo y distribución de los errores cometidos entre clases.
- Frecuencia de selección de cada característica en las 10 corridas con F_2 , junto con la media, desvío estándar, máximo, mínimo y mediana del número de características activas K .
- Analice cualitativamente la similaridad entre las soluciones encontradas en las 10 corridas con F_1 y en las 10 corridas con F_2 : ¿las corridas tienden a converger a arquitecturas similares o se observa alta variabilidad entre ellas? ¿Qué diferencias cualitativas se aprecian entre las soluciones obtenidas con cada función de aptitud? Justifique lo observado en términos de las propiedades de cada criterio de búsqueda.

Discuta si el AG exhibe un comportamiento estable o si las soluciones presentan alta variabilidad entre corridas.